

Coding Agent

Un agente de código autónomo impulsado por un LLM

Plan Mode

Supervisión humana

5 herramientas

OpenAI · Tavily

Autores: Marco Schenker · Ignacio Gaspar

Curso: Advanced Coding Agent

Fecha: 24 de junio de 2026

1. Resumen del proyecto

Este proyecto implementa un **agente de código** impulsado por un modelo de lenguaje (OpenAI) capaz de operar de forma autónoma sobre un repositorio. A partir de instrucciones en lenguaje natural, el agente puede:

- **Clonar y explorar** repositorios de GitHub.
- **Leer, analizar y modificar** archivos de código.
- **Ejecutar tareas autónomamente** mediante un bucle de razonamiento-acción (el modelo decide qué herramienta llamar y con qué argumentos).

El diseño combina dos mecanismos de control humano que distinguen a un agente seguro de uno que actúa a ciegas: un **modo de planificación** (el agente propone un plan que el usuario aprueba antes de tocar nada) y un **modo de supervisión** (las acciones destructivas requieren autorización explícita).

2. Arquitectura y funcionamiento

El agente sigue el patrón clásico de un bucle agéntico (*agentic loop*): el modelo recibe el historial de la conversación, decide si necesita una herramienta, el harness ejecuta esa herramienta, le devuelve el resultado, y el ciclo se repite hasta que el modelo produce una respuesta final o se alcanza el tope de iteraciones.

2.1 Las cinco herramientas (tools)

Herramienta	Función	¿Requiere autorización?
<code>read_file</code>	Lee el contenido de un archivo.	No
<code>write_file</code>	Escribe/crea un archivo (y sus carpetas).	Sí
<code>run_command</code>	Ejecuta un comando de shell (stdout + stderr).	Sí
<code>list_files</code>	Lista los archivos de un directorio.	No
<code>web_search</code>	Busca en la web vía la API de Tavily.	No

Cada herramienta se declara dos veces: como función de Python ejecutable y como *schema* JSON que se pasa al modelo en el parámetro `tools`, de modo que el LLM sepa qué argumentos espera cada una.

2.2 El bucle del harness

- **Fase de planificación** (si `plan_mode=True`): se pide al modelo un plan detallado *sin* herramientas. El usuario lo aprueba (`s`), lo rechaza o lo comenta. Un comentario reinicia la planificación con feedback.
- **Fase de ejecución**: hasta `max_iterations` pasos. En cada paso el modelo puede emitir una o más *tool_calls*; el harness las ejecuta (pidiendo autorización para `write_file` / `run_command` si la

supervisión está activa) y devuelve el resultado al modelo. Cuando el modelo responde sin `tool_calls`, esa es la respuesta final y el bucle termina.

- **Guardas de seguridad:** el tope de iteraciones evita bucles infinitos; la supervisión evita escrituras o comandos no deseados.

Correcciones aplicadas frente al notebook original. Se reprodujo la lógica original pero se corrigieron cuatro detalles: (1) la última línea `start_coding_agent(ç)` contenía un carácter inválido (`ç`) que provocaba un `NameError` — ahora es una llamada válida bajo `if __name__ == "__main__":`; (2) el cliente OpenAI se crea de forma perezosa (`get_client()`) para poder importar y testear el módulo sin clave; (3) los mensajes *assistant* con `tool_calls` se reconstruyen como diccionarios portables en el historial en vez de guardar el objeto crudo de la SDK; (4) `write_file` crea los directorios padre y `web_search` incluye el fragmento de contenido.

3. Implementación (código fuente)

Archivo: `coding_agent.py` — versión corregida y ejecutable.

```
"""
Coding Agent
=====
Un agente de código impulsado por un LLM (OpenAI) capaz de:
- Clonar y explorar repositorios de GitHub.
- Leer, analizar y modificar archivos de código.
- Ejecutar tareas de forma autónoma a partir de instrucciones en lenguaje natural.

Características del harness:
- Plan Mode: el agente propone primero un plan (sin usar herramientas) que el
  usuario aprueba antes de la ejecución.
- Supervision Mode: las herramientas sensibles (write_file, run_command)
  requieren autorización explícita del usuario antes de ejecutarse.
- Bucle agéntico con tope de iteraciones (max_iterations) para evitar bucles
  infinitos.

Autores: Marco Schenker, Ignacio Gaspar.

Nota sobre correcciones aplicadas frente al notebook original:
1. La última línea `start_coding_agent(ç)` contenía un carácter inválido (`ç`)
  que provocaba un NameError. Se reemplaza por una llamada válida bajo el
  guardia `if __name__ == "__main__":`.
2. Se añade un parámetro `model` configurable y se documenta la dependencia de
  las variables de entorno OPENAI_API_KEY y TAVILY_API_KEY.
3. Se evita serializar el objeto `msg` de OpenAI directamente en el historial:
  se reconstruye un dict con el rol "assistant" y sus tool_calls, lo que hace
  el historial portable y reproducible.
4. web_search usa el campo `content` (más informativo) además del título/URL.
"""

import os
import json
import subprocess

from openai import OpenAI

# =====
# CONFIGURACIÓN DEL CLIENTE
# =====
# Las claves se leen del entorno. Exporta antes de ejecutar:
# export OPENAI_API_KEY="sk-..."
# export TAVILY_API_KEY="tvly-..."
MODEL = os.environ.get("AGENT_MODEL", "gpt-5-nano")

# El cliente se crea de forma perezosa para que el módulo pueda importarse y
# probarse sin una clave válida ni acceso de red.
_client = None

def get_client():
    """Devuelve un cliente OpenAI, creándolo la primera vez que se usa."""
    global _client
    if _client is None:
        _client = OpenAI(api_key=os.environ.get("OPENAI_API_KEY"))
    return _client

# =====
# 1. DEFINICIÓN DE HERRAMIENTAS (TOOLS)
# =====

def read_file(filepath):
```

```

"""Lee el contenido de un archivo."""
try:
    with open(filepath, "r", encoding="utf-8") as f:
        return f.read()
except Exception as e:
    return f"Error leyendo archivo: {e}"

def write_file(filepath, content):
    """Escribe contenido en un archivo (crea directorios padre si hace falta)."""
    try:
        parent = os.path.dirname(filepath)
        if parent:
            os.makedirs(parent, exist_ok=True)
        with open(filepath, "w", encoding="utf-8") as f:
            f.write(content)
        return f"Éxito: Archivo {filepath} modificado."
    except Exception as e:
        return f"Error escribiendo archivo: {e}"

def run_command(command):
    """Ejecuta un comando de terminal y devuelve stdout + stderr."""
    try:
        result = subprocess.run(
            command, shell=True, capture_output=True, text=True
        )
        output = result.stdout
        if result.stderr:
            output += f"\nError/Stderr: {result.stderr}"
        return output if output.strip() else "Comando ejecutado con éxito (sin salida)."
    except Exception as e:
        return f"Error ejecutando comando: {e}"

def list_files(directory="."):
    """Lista los archivos en un directorio."""
    try:
        return "\n".join(sorted(os.listdir(directory)))
    except Exception as e:
        return f"Error listando directorio: {e}"

def web_search(query):
    """Busca información en la web usando la API de Tavily."""
    import requests

    api_key = os.environ.get("TAVILY_API_KEY")
    if not api_key:
        return "Error: No TAVILY_API_KEY."
    url = "https://api.tavily.com/search"
    payload = {"api_key": api_key, "query": query, "max_results": 3}
    try:
        response = requests.post(url, json=payload, timeout=15)
        data = response.json()
        results = data.get("results", [])
        if not results:
            return "Sin resultados."
        lines = []
        for r in results:
            snippet = (r.get("content") or "").strip().replace("\n", " ")
            lines.append(f"- {r.get('title')} ({r.get('url')})\n {snippet[:300]}")
        return "\n".join(lines)
    except Exception as e:
        return f"Error: {e}"

available_tools = {
    "read_file": read_file,
    "write_file": write_file,
    "run_command": run_command,

```

```

    "list_files": list_files,
    "web_search": web_search,
}

tools_schema = [
    {"type": "function", "function": {"name": "read_file", "description": "Lee un archivo.",
    "parameters": {"type": "object", "properties": {"filepath": {"type": "string"}},
    "required": ["filepath"]}}},
    {"type": "function", "function": {"name": "write_file", "description": "Escribe un archivo.",
    "parameters": {"type": "object", "properties": {"filepath": {"type": "string"},
    "content": {"type": "string"}}, "required": ["filepath", "content"]}}},
    {"type": "function", "function": {"name": "run_command", "description": "Ejecuta comando bash.",
    "parameters": {"type": "object", "properties": {"command": {"type": "string"}},
    "required": ["command"]}}},
    {"type": "function", "function": {"name": "list_files", "description": "Lista archivos.",
    "parameters": {"type": "object", "properties": {"directory": {"type": "string"}}}},
    {"type": "function", "function": {"name": "web_search", "description": "Busca en la web.",
    "parameters": {"type": "object", "properties": {"query": {"type": "string"}},
    "required": ["query"]}}}
]

# =====
# 2. EL HARNESS DEL AGENTE
# =====

SYSTEM_PROMPT = (
    "Eres un Agente de Código. Si plan_mode está activo, primero generarás un "
    "plan sin usar herramientas. Una vez aprobado, ejecutarás las herramientas "
    "necesarias para completar la tarea."
)

def start_coding_agent(plan_mode=True, supervision_mode=True, max_iterations=10):
    """Bucle interactivo del agente.

    Args:
        plan_mode: si es True, el agente propone un plan que debe aprobarse.
        supervision_mode: si es True, write_file y run_command piden autorización.
        max_iterations: tope de pasos de la fase de ejecución.
    """
    messages = [{"role": "system", "content": SYSTEM_PROMPT}]
    print(f"Agent Started. Plan Mode: {plan_mode} | "
          f"Supervisión: {supervision_mode} | Max Iters: {max_iterations}")

    while True:
        user_input = input("\n Tú: ")
        if user_input.lower() in ["salir", "exit"]:
            break
        messages.append({"role": "user", "content": user_input})

        # ----- FASE DE PLANIFICACIÓN -----
        if plan_mode:
            try:
                print("\n[Pensando Plan...]")
                res = get_client().chat.completions.create(
                    model=MODEL,
                    messages=messages + [{"role": "system",
                    "content": "Genera un plan detallado. NO uses herramientas todavía."}],
                )
                plan = res.choices[0].message.content
                print(f"\n PLAN:\n{plan}")

                feedback = input("\n ¿Aprobar? (s/n/comentario): ")
                if feedback.lower() != "s":
                    messages.append({"role": "assistant", "content": plan})
                    messages.append({"role": "user",
                    "content": f"Plan rechazado/modificado: {feedback}"})
                    continue

                messages.append({"role": "assistant", "content": plan})

```

```

        messages.append({"role": "user",
            "content": "Plan aprobado. PROCEDE CON LA EJECUCIÓN USANDO HERRAMIENTAS."})
    except Exception as e:
        print(f"Error: {e}")
        continue

# ----- FASE DE EJECUCIÓN -----
for i in range(1, max_iterations + 1):
    print(f"\n[Log] Iteración {i}/{max_iterations}...")
    try:
        response = get_client().chat.completions.create(
            model=MODEL, messages=messages, tools=tools_schema
        )
        msg = response.choices[0].message

        # Sin tool_calls -> respuesta final del agente.
        if not msg.tool_calls:
            messages.append({"role": "assistant", "content": msg.content})
            print(f"\n Agente: {msg.content}")
            break

        # Reconstruimos el mensaje assistant con sus tool_calls (portable).
        messages.append({
            "role": "assistant",
            "content": msg.content,
            "tool_calls": [{
                "id": tc.id,
                "type": "function",
                "function": {"name": tc.function.name,
                    "arguments": tc.function.arguments},
            } for tc in msg.tool_calls],
        })

        for tool_call in msg.tool_calls:
            name = tool_call.function.name
            args = json.loads(tool_call.function.arguments)
            print(f"Tool: {name}({args})")

            if supervision_mode and name in ["write_file", "run_command"]:
                if input(f"△ ¿Autorizar {name}? (s/n): ").lower() != "s":
                    result = "Error: Usuario denegó ejecución."
                else:
                    result = available_tools[name](**args)
            else:
                result = available_tools[name](**args)

            messages.append({
                "role": "tool",
                "tool_call_id": tool_call.id,
                "name": name,
                "content": str(result),
            })
        except Exception as e:
            print(f" Error: {e}")
            break
    else:
        print("△ Límite alcanzado.")

if __name__ == "__main__":
    start_coding_agent(plan_mode=True, supervision_mode=True, max_iterations=10)

```

4. Ejemplos de ejecución

Las siguientes transcripciones provienen de corridas reales del agente y muestran los distintos escenarios pedidos en la consigna.

4.1 Explorar el repositorio y resumir el README

```
□ Coding Agent Iniciado. | Plan Mode: Activado | Supervisión: Activada

□ Tú: Lista los archivos y carpetas del directorio actual. Luego, lee el archivo README.md y armame un resumen en 3 viñetas sobre el propósito del repositorio.

□ Agente: Plan de acción propuesto:
- Paso 1: Listar los archivos y carpetas del directorio actual.
- Paso 2: Verificar si existe README.md y, de ser así, leer su contenido.
- Paso 3: Generar un resumen en 3 viñetas sobre el propósito del repositorio.
- Paso 4: Entregar el listado, el contenido leído y el resumen.
¿Apruebas este plan para proceder?

□ Tú: sí

< list_files({'directory': '.'})
< read_file({'filepath': 'README.md'})

□ Agente: Plan implementado y resultados:
1) Listado: AGENTS.md, README.md, images/, .github/, authors.yaml, articles/, registry.yaml, examples/, CONTRIBUTING.md, LICENSE, .gitignore
2) Lectura de README.md: "Example code and guides for accomplishing common tasks with the OpenAI API ... MIT License"
3) Resumen en 3 viñetas:
- Proporciona ejemplos de código y guías para tareas comunes con la OpenAI API.
- Explica cómo ejecutar los ejemplos (requiere API key, variable OPENAI_API_KEY o archivo .env); mayoritariamente Python.
- Repositorio con licencia MIT que agrupa recursos para aprender y experimentar con la OpenAI API.
```

4.2 Buscar información en la web (Wikipedia vía Tavily)

```
□ Coding Agent Iniciado. | Plan Mode: Activado | Supervisión: Activada

□ Tú: Contame la historia de Roma de Wikipedia

□ Agente: [propone un plan: definir alcance, fuente (Wikipedia), elaboración cronológica, presentación y atribución] ¿Apruebas este plan?

□ Tú: aprobado

□ Agente: [entrega resumen general + línea de tiempo con hitos:
753 a.C. fundación legendaria · 509 a.C. inicio República · Guerras Púnicas ·
27 a.C. Augusto e inicio del Imperio · Pax Romana · 313 Edicto de Milán ·
330 Constantinopla · 476 caída de Occidente · 1453 caída de Bizancio]
¿Quieres (a) resumen breve, (b) línea de tiempo detallada, (c) personajes,
(d) lecturas?

□ Tú: a

□ Agente: [entrega un resumen breve de ~2 párrafos sobre Monarquía, República, Imperio, Pax Romana, crisis, caída y legado de Roma]

(Nota: la herramienta web_search consulta la API de Tavily para obtener fuentes.)
```

4.3 Crear un archivo y ejecutarlo (Hello World)

```
□ Agent Started. Plan: True | Supervision: True | Max Iters: 10

□ Tú: Crea un nuevo archivo de código con un hello world

□ PLAN PROPUESTO: [el agente lista un plan extenso y pregunta qué lenguaje usar]

□ ¿Aprobar plan? un simple archivo python con un hello world para ejecutar en
  consola. Muy simple, aprobar

□ PLAN PROPUESTO: [plan adaptado a un hello_world.py en Python]
□ ¿Aprobar plan? s

[Log] Iteración 1/10...
✂ write_file({'filepath': 'hello_world.py',
              'content': '#!/usr/bin/env python3\nprint("Hello, World!")\n'})
△ ¿Autorizar write_file? s

[Log] Iteración 2/10...
✂ run_command({'command': 'python3 hello_world.py'})
△ ¿Autorizar run_command? s

[Log] Iteración 3/10...
□ Agente: Plan aprobado y ejecución completada.
  Archivo creado: hello_world.py
  Salida de ejecución: Hello, World!
```

4.4 Crear código + tests, refactorizar y verificar (factorial)

```
[ ] Agent Started. Plan Mode: True

[ ] Tú: Escribe prueba_agente.py con una función factorial(5) que imprima en consola.
      Ejecútalo y dime el output. Luego crea un archivo de tests aparte.
      Después refactoriza el código y revisa que los tests sigan pasando;
      si no pasan, refactoriza los tests.

[ ] PLAN: [crear prueba_agente.py con factorial pura + main; tests con pytest;
          ejecutar; refactorizar; re-ejecutar] [ ] ¿Aprobar? s

[Log] Iteración 1/10... < write_file('prueba_agente.py', <factorial iterativo + main>)
[Log] Iteración 2/10... < write_file('tests/test_prueba_agente.py', <4 tests pytest>)
[Log] Iteración 3/10... < run_command('mkdir -p tests')
[Log] Iteración 4/10... < write_file('tests/test_prueba_agente.py', ...)
[Log] Iteración 5/10... < run_command('pytest -q')
[Log] Iteración 6/10... < run_command('pytest -q tests/test_prueba_agente.py')
[Log] Iteración 7/10... < write_file(... fix de sys.path para imports ...)
[Log] Iteración 8/10... < run_command('pytest -q tests/test_prueba_agente.py')
[Log] Iteración 9/10... < run_command('python3 prueba_agente.py') -> 120
[Log] Iteración 10/10... < write_file(... amplía a 6 tests ...)
△ Límite alcanzado.

[ ] Tú: continua

[ ] PLAN: [completar refactorización: validación de tipo TypeError, docstrings,
          más tests] [ ] ¿Aprobar? s

[Log] Iteración 1/10... < write_file('prueba_agente.py', <+ TypeError + docstring>)
[Log] Iteración 2/10... < write_file('tests/test_prueba_agente.py', <7 tests>)
[Log] Iteración 3-5/10... < run_command('pytest -q') ... 'python3 prueba_agente.py' -> 120
[Log] Iteración 6/10... < read_file('prueba_agente.py')
[Log] Iteración 7/10...
[ ] Agente: Plan completado.
  - factorial(n): valida tipo (TypeError) y negativos (ValueError); cálculo
    iterativo; función pura; main imprime factorial(5).
  - 7 tests cubren 5,0,1,2,3, ValueError y TypeError. Todos pasan.
  - Ejecución del script: 120.
```

5. Respuestas a las preguntas

¿Cuántas iteraciones del loop hizo el agente?

En la primera fase (implementación inicial) el agente alcanzó el **límite de 10 iteraciones**. Esto ocurrió porque realizó múltiples intentos de escritura de archivos, creación de directorios y ejecución de tests para asegurar que el entorno estuviera correctamente configurado. En la segunda fase (refactorización) usó **7 iteraciones adicionales** para completar las mejoras y la validación final.

¿Qué salió bien y qué salió mal?

Bien: la separación de fases (plan vs. ejecución) funcionó correctamente. El agente logró escribir el código, configurar pytest con ajustes de `sys.path` para evitar errores de importación, y validar tanto el flujo principal como los casos de error.

Mal: el agente llegó al límite de iteraciones en la primera parte. Se debió a una redundancia al escribir el archivo de tests varias veces y a la verificación exhaustiva de rutas, lo que consumió pasos innecesarios.

¿Qué mejoraría?

- **Eficiencia de herramientas:** permitir realizar varias acciones (p. ej. crear un directorio y un archivo) en menos pasos o de forma combinada.
- **Contexto de rutas:** preconfigurar el `PYTHONPATH` para que el agente no gaste iteraciones resolviendo problemas de importación de módulos locales.
- **Detección de redundancia:** mejorar el prompt para que el agente verifique si un archivo ya tiene el contenido correcto antes de reescribirlo.

6. Cómo ejecutar

```
export OPENAI_API_KEY="sk-..."
export TAVILY_API_KEY="tvly-..."
pip install openai requests
python3 coding_agent.py
```

El agente arranca en modo interactivo. Escribe una instrucción en lenguaje natural; aprueba el plan con `s`; autoriza las escrituras/comandos cuando se soliciten. Escribe `salir` para terminar.